

CS1675/2075: Introduction to Machine Learning

Week 9: Feature Engineering and Non-Linear Basis Functions

Fall 2024

Quick review: fitting linear models

- From a Bayesian perspective, we mostly assumed infinitely diffuse priors on the unknown model coefficients, β .
 - Highlights the relationship between the MAP, the MLE, and OLS estimators.
- Most of the derivations we discussed assumed a fixed or known noise, σ .
 - Wanted to focus on the uncertainty on β and how that covariance structure is dictated by the covariance of the inputs via the sum of squares matrix, $\mathbf{X}^T \mathbf{X}$.

Bayesian fitting of linear models with informed priors revealed the same structure as the unknown constant mean!

- The posterior mean is a precision weighted average of the prior and the data!
- The posterior precision is the sum of the prior and data precisions.

When the noise is unknown, σ must be learned along with the $\boldsymbol{\beta}$ coefficients

- In a non-Bayesian setting, the maximum likelihood estimate (MLE) on the squared noise, $\hat{\sigma}^2$, requires the OLS estimates, $\hat{\boldsymbol{\beta}}$, to be determined.
- The MLE is proportional to the SSE evaluated at the OLS estimates divided by the number of samples.

$$\hat{\sigma}^2 \propto \frac{SSE(\hat{\boldsymbol{\beta}})}{N} = \frac{1}{N} (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})^T (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) = MSE$$

Written as proportional to because the formal definition for the MLE on σ^2 divides by N minus the degrees of freedom. Conceptually, the residual variance represents the MSE of the model.

The term **RESIDUAL VARIANCE** represents the variation the model **CANNOT** explain

In a Bayesian setting, we just have another unknown parameter to learn

- Need to specify the prior belief on the noise as well as the trend coefficients.

$$p(\boldsymbol{\beta}, \sigma \mid \mathbf{X}, \mathbf{y}) \propto p(\mathbf{y} \mid \mathbf{X}, \boldsymbol{\beta}, \sigma) \times p(\boldsymbol{\beta}, \sigma)$$

- I like to use independent priors:

$$p(\boldsymbol{\beta}, \sigma) = p(\boldsymbol{\beta})p(\sigma)$$

If we would use standard normal priors on the $\boldsymbol{\beta}$ parameters of a **single input linear relationship model** and an Exponential prior with rate equal to 1 for σ

$$p(y_n \mid \mu_n, \sigma) \sim \text{normal}(y_n \mid \mu_n, \sigma)$$

$$\mu_n = \beta_0 + \beta_1 x_n$$

$$\beta_0 \sim \text{normal}(\beta_0 \mid 0, 1)$$

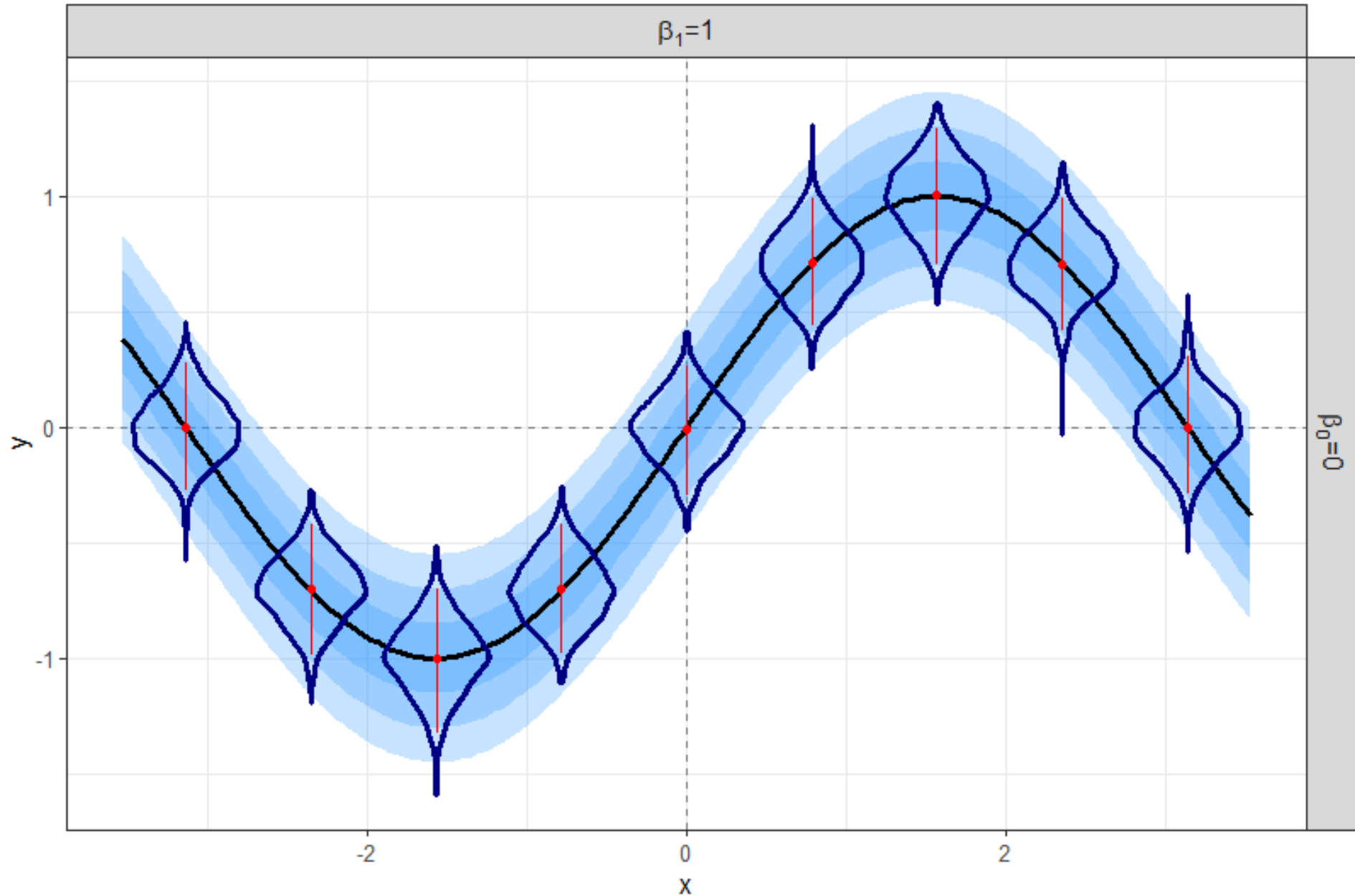
$$\beta_1 \sim \text{normal}(\beta_1 \mid 0, 1)$$

$$\sigma \mid \lambda = 1 \sim \text{Exp}(\sigma \mid \lambda = 1)$$

What does linearity mean?

- A linear model is NOT linear because of a linear relationship between input and mean trend.
- A linear model is linear because the unknown coefficients are linearly related to the mean trend!

We saw that the Gaussian likelihood sampling distribution still holds even with a sine wave!



Allows us to extend the linear model to handle complex modeling tasks!

- In fact, when we first introduced regression problems and complexity we worked through a **polynomial basis**.
- Rather than denoting the features to the design matrix as the inputs x_d , we can denote the features as some basis feature, ϕ_j .

J -th degree polynomial basis applied to a single input x

- The j -th basis **FEATURE** is: $\phi_j = x^j$.
- The 0-th **FEATURE** is just the intercept: $\phi_0 = x^0 = 1$.
- The mean trend of a polynomial basis is:

$$\mu_n = \beta_0 + \sum_{j=1}^J (\beta_j \phi_{n,j}) = \sum_{j=0}^J (\beta_j \phi_{n,j}) = \sum_{j=0}^J (\beta_j x_n^j)$$

An earlier homework involved comparing a 1st degree polynomial up to 9th degree!

$$\text{Linear } \mu_n = \sum_{j=0}^{J=1} (\beta_j \phi_{n,j}) = \sum_{j=0}^{J=1} (\beta_j x_n^j) = \beta_0 x_n^{j=0} + \beta_1 x_n^{j=1} = \beta_0 + \beta_1 x_n$$

An earlier homework involved comparing a 1st degree polynomial up to 9th degree!

Linear $\mu_n = \sum_{j=0}^{J=1} (\beta_j \phi_{n,j}) = \sum_{j=0}^{J=1} (\beta_j x_n^j) = \beta_0 x_n^{j=0} + \beta_1 x_n^{j=1} = \beta_0 + \beta_1 x_n$

Quadratic $\mu_n = \sum_{j=0}^{J=2} (\beta_j \phi_{n,j}) = \sum_{j=0}^{J=2} (\beta_j x_n^j) = \beta_0 x_n^{j=0} + \beta_1 x_n^{j=1} + \beta_2 x_n^{j=2} = \beta_0 + \beta_1 x_n + \beta_2 x_n^2$

An earlier homework involved comparing a 1st degree polynomial up to 9th degree!

Linear $\mu_n = \sum_{j=0}^{J=1} (\beta_j \phi_{n,j}) = \sum_{j=0}^{J=1} (\beta_j x_n^j) = \beta_0 x_n^{j=0} + \beta_1 x_n^{j=1} = \beta_0 + \beta_1 x_n$

Quadratic $\mu_n = \sum_{j=0}^{J=2} (\beta_j \phi_{n,j}) = \sum_{j=0}^{J=2} (\beta_j x_n^j) = \beta_0 x_n^{j=0} + \beta_1 x_n^{j=1} + \beta_2 x_n^{j=2} = \beta_0 + \beta_1 x_n + \beta_2 x_n^2$

$$\mu_n = \sum_{j=0}^{J=8} (\beta_j \phi_{n,j}) = \sum_{j=0}^{J=8} (\beta_j x_n^j) = \beta_0 x_n^{j=0} + \beta_1 x_n^{j=1} + \beta_2 x_n^{j=2} + \cdots + \beta_7 x_n^{j=7} + \beta_8 x_n^{j=8}$$

8th degree
polynomial

$$\mu_n = \sum_{j=0}^{J=8} (\beta_j \phi_{n,j}) = \beta_0 + \beta_1 x_n^1 + \beta_2 x_n^2 + \cdots + \beta_7 x_n^7 + \beta_8 x_n^8$$

The basis design matrix is still structured as:
rows are observations and columns are *features*

$$\Phi = \begin{bmatrix} \phi_{1,0} & \phi_{1,1} & \cdots & \phi_{1,J} \\ \phi_{2,0} & \phi_{2,1} & \cdots & \phi_{2,J} \\ \vdots & \vdots & \ddots & \vdots \\ \phi_{N,0} & \phi_{N,1} & \cdots & \phi_{N,J} \end{bmatrix}$$

For the polynomial basis, each column is the input raised to a power

$$\Phi = \begin{bmatrix} \phi_{1,0} & \phi_{1,1} & \cdots & \phi_{1,J} \\ \phi_{2,0} & \phi_{2,1} & \cdots & \phi_{2,J} \\ \vdots & \vdots & \ddots & \vdots \\ \phi_{N,0} & \phi_{N,1} & \cdots & \phi_{N,J} \end{bmatrix} = \begin{bmatrix} 1 & x_1 & \cdots & x_1^J \\ 1 & x_2 & \cdots & x_2^J \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & \cdots & x_N^J \end{bmatrix}$$

The mean trend vector for a linear basis model is just matrix multiplication of the basis design matrix and the coefficient vector!

- Store the $J + 1$ unknown coefficients in a column vector $\boldsymbol{\beta}$.
- The mean trend vector is calculated simply as:

$$\boldsymbol{\mu} = \boldsymbol{\Phi} \boldsymbol{\beta}$$

The log-likelihood for N observations when the noise is known is proportional to...

$$\log[p(\mathbf{y} \mid \Phi, \boldsymbol{\beta}, \sigma)] \propto -\frac{1}{2\sigma^2} (\mathbf{y} - \Phi\boldsymbol{\beta})^T (\mathbf{y} - \Phi\boldsymbol{\beta})$$

The formula for the Maximum Likelihood Estimates (MLEs)...is the same!

$$\boldsymbol{\beta}_{MLE} = (\boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^T \mathbf{y}$$

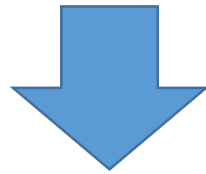
The formula for the Maximum Likelihood Estimates (MLEs)...is the same!

$$\boldsymbol{\beta}_{MLE} = (\boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^T \mathbf{y}$$

Simply replace **X** with **Φ**, everything is the same!

In a Bayesian analysis, with known σ and an infinitely diffuse prior on the coefficients...

$$p(\boldsymbol{\beta} \mid \boldsymbol{\Phi}, \mathbf{y}, \sigma) \propto p(\mathbf{y} \mid \boldsymbol{\Phi}, \boldsymbol{\beta}, \sigma)p(\boldsymbol{\beta})$$



$$p(\boldsymbol{\beta} \mid \boldsymbol{\Phi}, \mathbf{y}, \sigma) \propto p(\mathbf{y} \mid \boldsymbol{\Phi}, \boldsymbol{\beta}, \sigma)$$

In a Bayesian analysis, with known σ and an infinitely diffuse prior on the coefficients...

The posterior is still MVN! Just replace $\mathbf{X}$ with Φ :

$$\boldsymbol{\beta} \mid \Phi, \mathbf{y}, \sigma \sim \mathcal{N} \left(\boldsymbol{\beta} \mid \hat{\boldsymbol{\beta}}, \text{cov}(\boldsymbol{\beta}, \boldsymbol{\beta}) \right)$$

$$\boldsymbol{\beta} \mid \Phi, \mathbf{y}, \sigma \sim \mathcal{N}(\boldsymbol{\beta} \mid (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}, \sigma^2 (\Phi^T \Phi)^{-1})$$

As already discussed, we do not have to just consider a polynomial basis

- We can use many different types of basis functions.
- Consider the simple parametric sine wave for a single input:

$$\mu_n = \beta_0 + \beta_1 \sin(x_n)$$

The basis design matrix evaluates the $\sin(x_n)$ at each observation

$$\Phi = \begin{bmatrix} 1 & \sin(x_1) \\ 1 & \sin(x_2) \\ 1 & \sin(x_3) \\ \vdots & \vdots \\ 1 & \sin(x_N) \end{bmatrix}$$

The unknown coefficients are still stored in a column vector

$$\boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}$$

- The number of rows of the β -parameter column vector must equal the number of columns of the basis design matrix

The mean trend vector still calculated via matrix multiplication!

$$\boldsymbol{\mu} = \boldsymbol{\Phi}\boldsymbol{\beta} = \begin{bmatrix} 1 & \sin(x_1) \\ 1 & \sin(x_2) \\ 1 & \sin(x_3) \\ \vdots & \vdots \\ 1 & \sin(x_N) \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} = \begin{bmatrix} \beta_0 + \beta_1 \sin(x_1) \\ \beta_0 + \beta_1 \sin(x_2) \\ \beta_0 + \beta_1 \sin(x_3) \\ \vdots \\ \beta_0 + \beta_1 \sin(x_N) \end{bmatrix}$$

Let's see how to do this in R

- First create some random observations around the true sine wave.

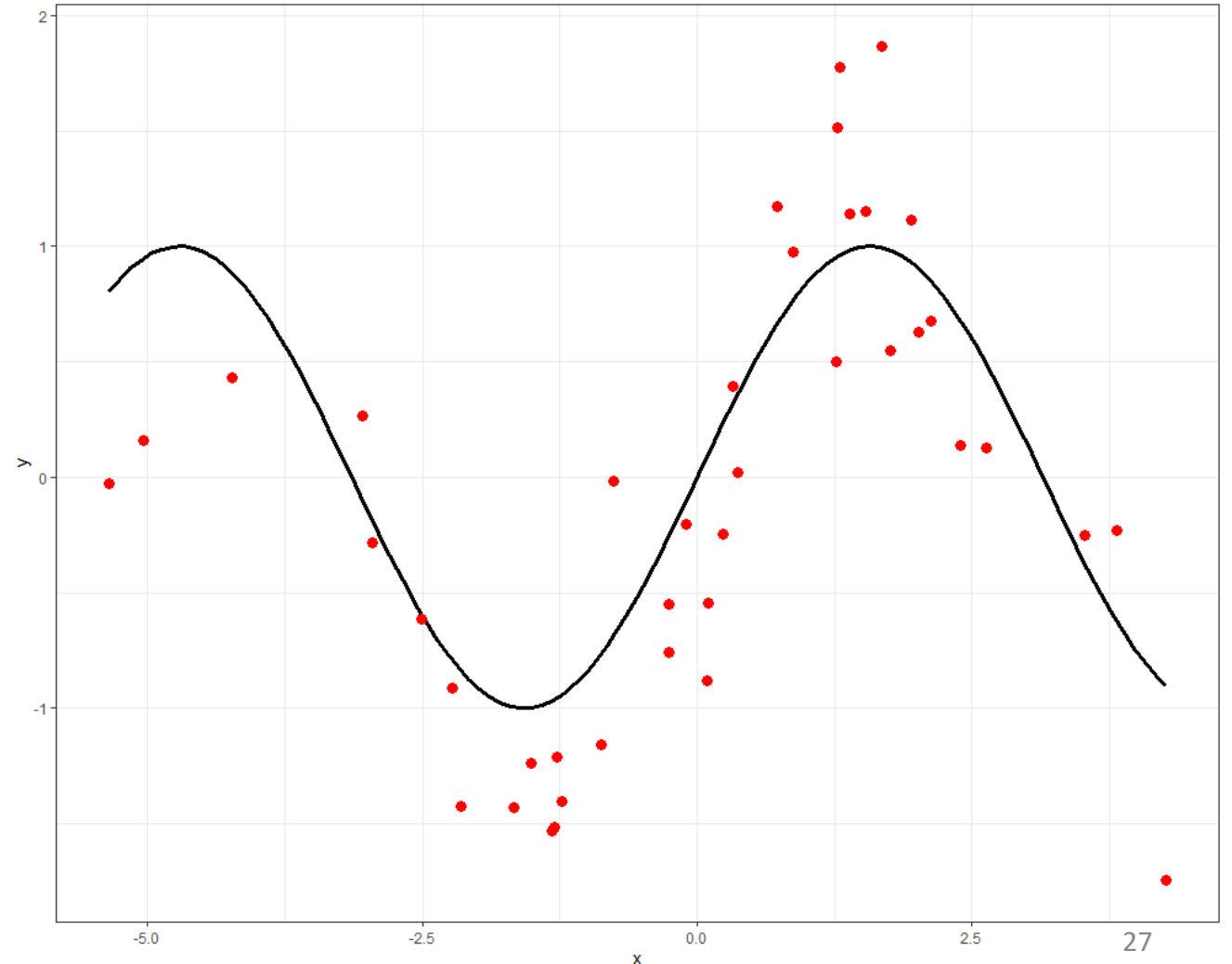
```
set.seed(32123)
demo_sine <- data.frame(x = rnorm(n = 41, mean = 0, sd = 2)) %>%
  mutate(y = rnorm(n = n(), mean = sin(x), sd = 0.45))
```

```
> demo_sine %>% head()
      x      y
1 -0.87107896 -0.8957159
2  0.09133065 -0.2332361
3 -2.14708880 -1.0341372
4  1.29583553  1.2338161
5 -1.22543156 -1.0941230
6  2.39978593  0.4957598
```

Data set contains 2 columns one for the input and one for the noisy response

Random observations displayed as red markers around the true sine wave in black

```
demo_sine %>%  
  ggplot(mapping = aes(x = x, y = y)) +  
    stat_function(fun = sin,  
                 color = "black",  
                 size = 1.15) +  
    geom_point(size = 3, color = "red") +  
    theme_bw()
```



Create the design matrix using the `model.matrix()` function

```
> Xsine <- model.matrix(y ~ sin(x), data = demo_sine)
> class(Xsine)
[1] "matrix" "array"
> dim(Xsine)
[1] 41  2
> head(Xsine)
      (Intercept)      sin(x)
1              1 -0.76502423
2              1  0.09120374
3              1 -0.83848871
4              1  0.96243584
5              1 -0.94095203
6              1  0.67562102
```

Create the design matrix using the `model.matrix()` function

```
> Xsine <- model.matrix(y ~ sin(x), data = demo_sine)
> class(Xsine)
[1] "matrix" "array"
> dim(Xsine)
[1] 41  2
> head(Xsine)
      (Intercept)      sin(x)
1              1 -0.76502423
2              1  0.09120374
3              1 -0.83848871
4              1  0.96243584
5              1 -0.94095203
6              1  0.67562102
```

Formula reads that
“ y is a function of
 $\sin(x)$ ”

Store the β -parameters as a regular vector and convert to a column vector with `as.matrix()`

```
> ### the column vector of the intercept and slope
> bsine <- c(0, 1)
> as.matrix(bsine)
      [,1]
[1,]    0
[2,]    1
> mu_sine <- Xsine %*% as.matrix(bsine)
> class(mu_sine)
[1] "matrix" "array"
> dim(mu_sine)
[1] 41  1
> head(mu_sine)
      [,1]
1 -0.76502423
2  0.09120374
3 -0.83848871
4  0.96243584
5 -0.94095203
6  0.67562102
> # convert trend matrix data type to vector data type
> as.vector(mu_sine) %>% head()
[1] -0.76502423  0.09120374 -0.83848871  0.96243584 -0.94095203  0.67562102
```

Store the β -parameters as a regular vector and convert to a column vector with `as.matrix()`

```
> ### the column vector of the intercept and slope
> bsine <- c(0, 1) Assign a numeric "regular vector" to the variable bsine
> as.matrix(bsine)
      [,1]
[1,]    0
[2,]    1
> mu_sine <- Xsine %*% as.matrix(bsine)
> class(mu_sine)
[1] "matrix" "array"
> dim(mu_sine)
[1] 41  1
> head(mu_sine)
      [,1]
1 -0.76502423
2  0.09120374
3 -0.83848871
4  0.96243584
5 -0.94095203
6  0.67562102
> # convert trend matrix data type to vector data type
> as.vector(mu_sine) %>% head()
[1] -0.76502423  0.09120374 -0.83848871  0.96243584 -0.94095203  0.67562102
```

Store the β -parameters as a regular vector and convert to a column vector with `as.matrix()`

```
> ### the column vector of the intercept and slope
```

```
> bsine <- c(0, 1)
```

```
> as.matrix(bsine)
```

```
      [,1]
```

```
[1,]    0
```

```
[2,]    1
```

Use the data type conversion function, `as.matrix()`, to put the coefficients into the correct column vector format

```
> mu_sine <- xsine %*% as.matrix(bsine)
```

```
> class(mu_sine)
```

```
[1] "matrix" "array"
```

```
> dim(mu_sine)
```

```
[1] 41  1
```

```
> head(mu_sine)
```

```
      [,1]
```

```
1 -0.76502423
```

```
2  0.09120374
```

```
3 -0.83848871
```

```
4  0.96243584
```

```
5 -0.94095203
```

```
6  0.67562102
```

```
> # convert trend matrix data type to vector data type
```

```
> as.vector(mu_sine) %>% head()
```

```
[1] -0.76502423  0.09120374 -0.83848871  0.96243584 -0.94095203  0.67562102
```


Store the β -parameters as a regular vector and convert to a column vector with `as.matrix()`

```
> ### the column vector of the intercept and slope
> bsine <- c(0, 1)
> as.matrix(bsine)
      [,1]
[1,]    0
[2,]    1
```

```
> mu_sine <- Xsine %*% as.matrix(bsine)
```

```
> class(mu_sine)
[1] "matrix" "array"
> dim(mu_sine)
[1] 41  1
> head(mu_sine)
      [,1]
1 -0.76502423
2  0.09120374
3 -0.83848871
4  0.96243584
5 -0.94095203
6  0.67562102
```

```
> # convert trend matrix data type to vector data type
> as.vector(mu_sine) %>% head()
```

```
[1] -0.76502423  0.09120374 -0.83848871  0.96243584 -0.94095203  0.67562102
```

Matrix multiply the basis design matrix with the coefficient column vector

Store the β -parameters as a regular vector and convert to a column vector with `as.matrix()`

```
> ### the column vector of the intercept and slope
> bsine <- c(0, 1)
> as.matrix(bsine)
```

```
      [,1]
[1,]     0
[2,]     1
```

```
> mu_sine <- Xsine %*% as.matrix(bsine)
```

```
> class(mu_sine)
```

```
[1] "matrix" "array"
```

```
> dim(mu_sine)
```

```
[1] 41  1
```

```
> head(mu_sine)
```

```
      [,1]
1 -0.76502423
2  0.09120374
3 -0.83848871
4  0.96243584
5 -0.94095203
6  0.67562102
```

The result is a column vector (matrix data type)

```
> # convert trend matrix data type to vector data type
```

```
> as.vector(mu_sine) %>% head()
```

```
[1] -0.76502423  0.09120374 -0.83848871  0.96243584 -0.94095203  0.67562102
```

Store the β -parameters as a regular vector and convert to a column vector with `as.matrix()`

```
> ### the column vector of the intercept and slope
> bsine <- c(0, 1)
> as.matrix(bsine)
      [,1]
[1,]    0
[2,]    1
> mu_sine <- Xsine %*% as.matrix(bsine)
> class(mu_sine)
[1] "matrix" "array"
> dim(mu_sine)
[1] 41  1
> head(mu_sine)
      [,1]
1 -0.76502423
2  0.09120374
3 -0.83848871
4  0.96243584
5 -0.94095203
6  0.67562102
```

Convert to a “regular” numeric vector with the `as.vector()` function (could also use `as.numeric()`)

```
> # convert trend matrix data type to vector data type
> as.vector(mu_sine) %>% head()
[1] -0.76502423  0.09120374 -0.83848871  0.96243584 -0.94095203  0.67562102
```

Fit a linear basis model the same way we fit any linear model!

- Non-Bayesian...find the regression coefficients that minimize the SSE!

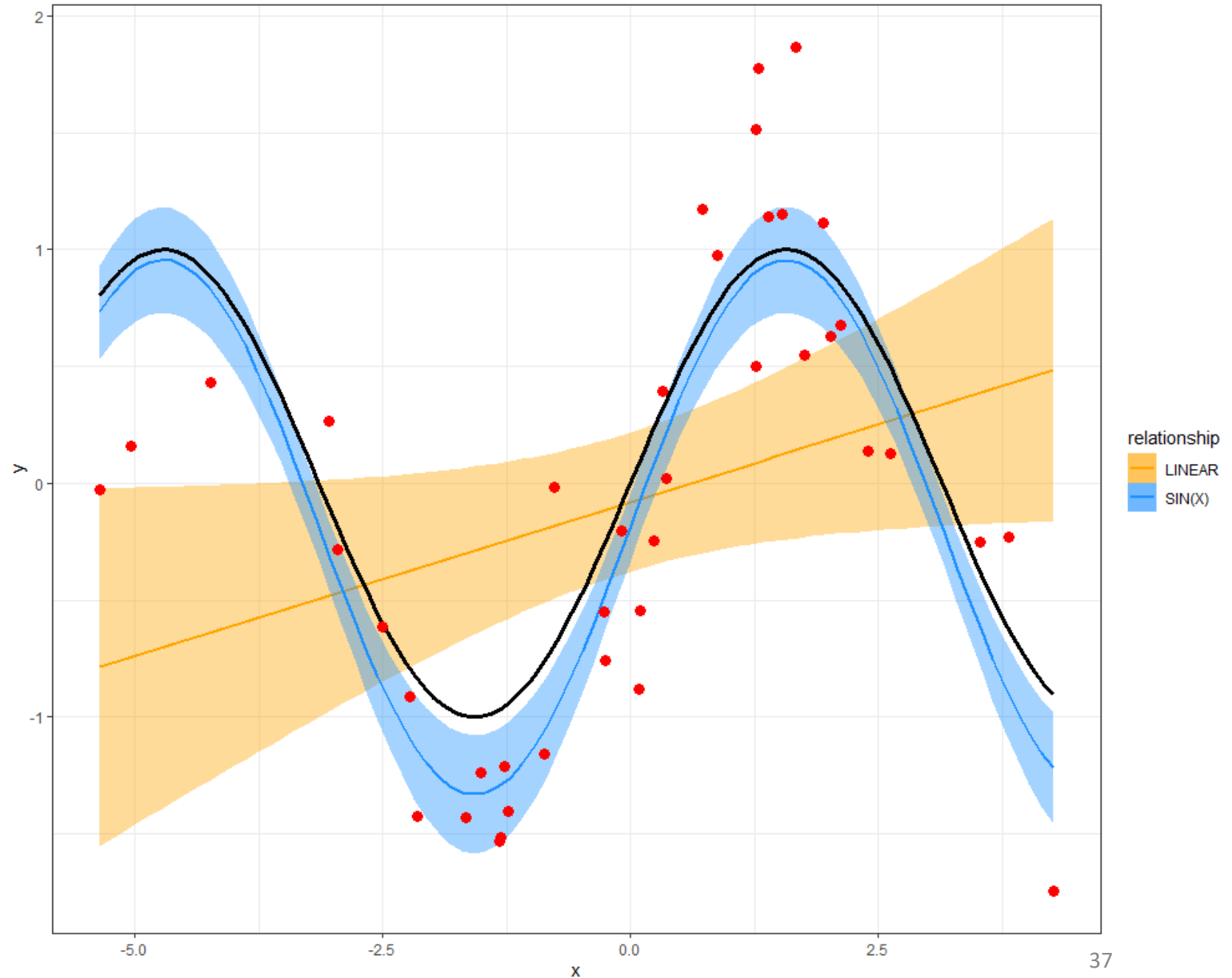
```
> ### fit the sine wave model!  
> sine_mod_mle <- lm( y ~ sin(x), data = demo_sine )  
> ### coefficient summary  
> sine_mod_mle %>% broom::tidy()  
# A tibble: 2 × 5  
  term          estimate std.error statistic  p.value  
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>  
1 (Intercept)  -0.187      0.0714     -2.61 1.26e- 2  
2 sin(x)        1.14       0.0946     12.1 9.32e-15
```

- The unknown regression coefficients are LINEARLY related to the TREND even though the FEATURE is a non-linear function of the INPUT!!!

Linear models are so much **MORE** than just “*best fit lines*”!!!!!!!

Just because a LINEAR relationship does NOT explain the data...you SHOULD **NOT** jump straight to training neural networks or gradient boosted trees!

Spend the time to try several types of basis functions to capture non-linear relationships!



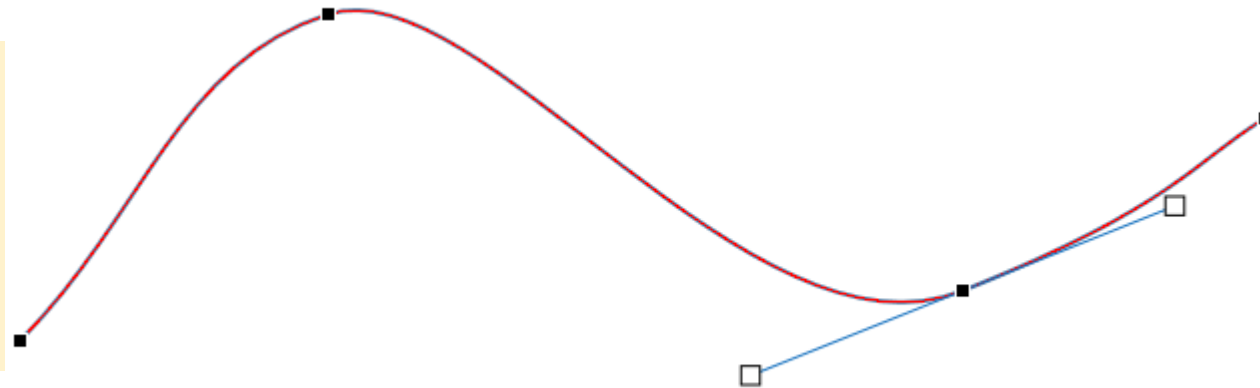
You might be thinking...

- “You KNEW the basis function to try...you KNEW to use $\sin(x)$...what if we DON'T KNOW what to try!?!?!”

“Build” complex non-linear relationships from simple polynomials...

- **SPLINES** are piecewise polynomials, which “build up” complicated relationships by “stitching together” low order polynomials.
- You have probably worked with splines before!

Splines are common tools for drawing smooth curves in graphics and computer animation!



There are many different flavors of SPLINES

- Some common types:
 - Bezier splines
 - Cubic splines
 - B-splines
 - Natural Splines
 - Thin Plate Splines
 - Many more!!!
- All SPLINES are used to SMOOTH data and thus model NON-LINEAR output-to-input relationships!
- If you're interested to learn more about splines...
 - [This site](#) provides several examples of smoothing, 2D, and 3D splines!
 - Section 7.4 of ISL provides even more details on the construction of splines.

There are many different flavors of SPLINES

- Some common types:
 - Bezier splines
 - Cubic splines
 - B-splines
 - Natural Splines We will focus on Natural Splines but the concepts apply to other types!
 - Thin Plate Splines
 - Many more!!!
- All SPLINES are used to SMOOTH data and thus model NON-LINEAR output-to-input relationships!
- If you're interested to learn more about splines...
 - [This site](#) provides several examples of smoothing, 2D, and 3D splines!
 - Section 7.4 of ISL provides even more details on the construction of splines.

How do SPLINES work?

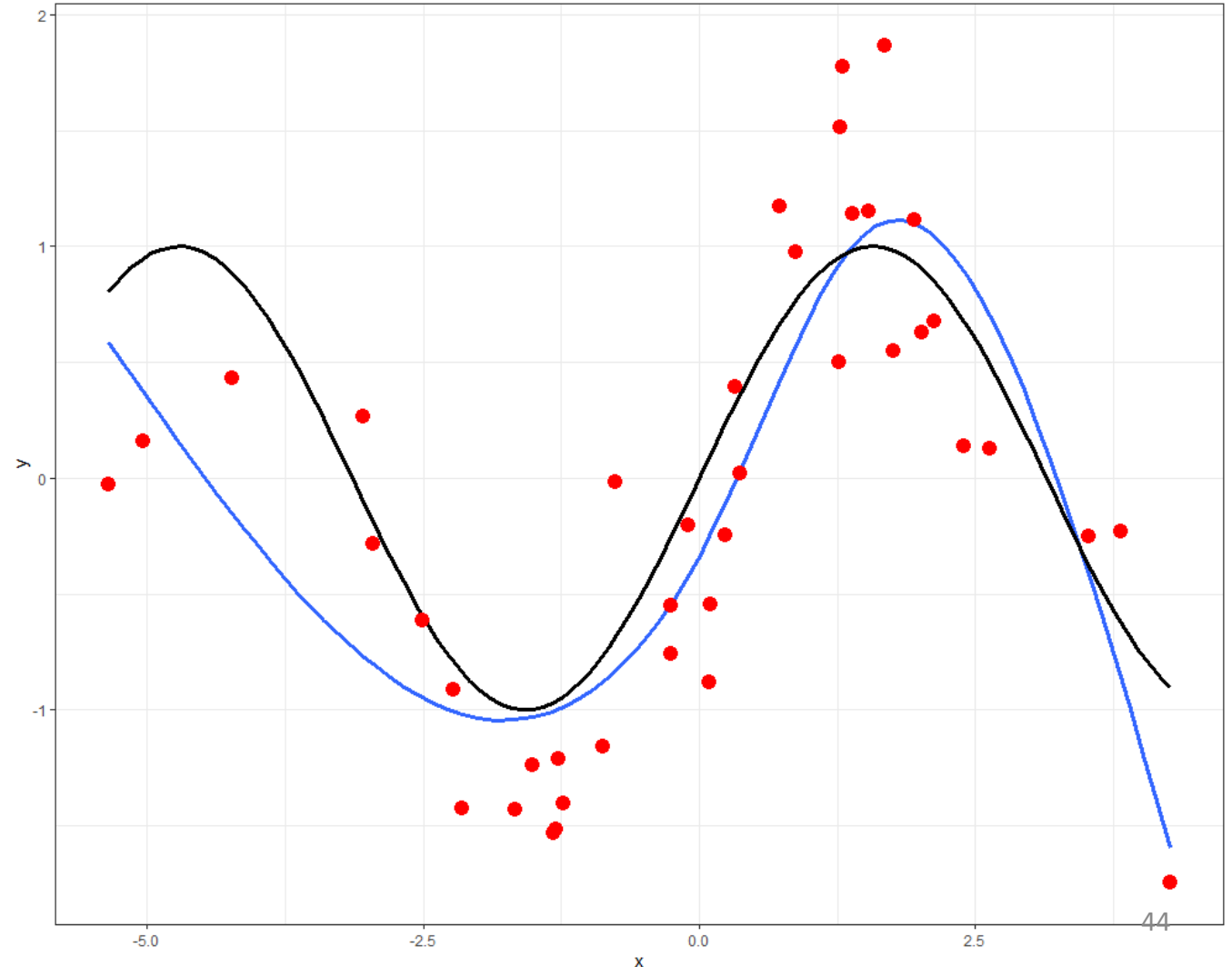
- SPLINES can be represented as LINEAR BASIS functions.
- The BASIS functions serve as non-linear FEATURES generated from the INPUT.
- The output is modeled as the **WEIGHTED SUM** of the non-linear FEATURES!

However, let's LOOK at the result of a natural spline linear basis model...before examining HOW the basis functions work...

- Natural spline basis is created with the `splines::ns()` function.
- The `splines` package comes with base R so everyone has it already!

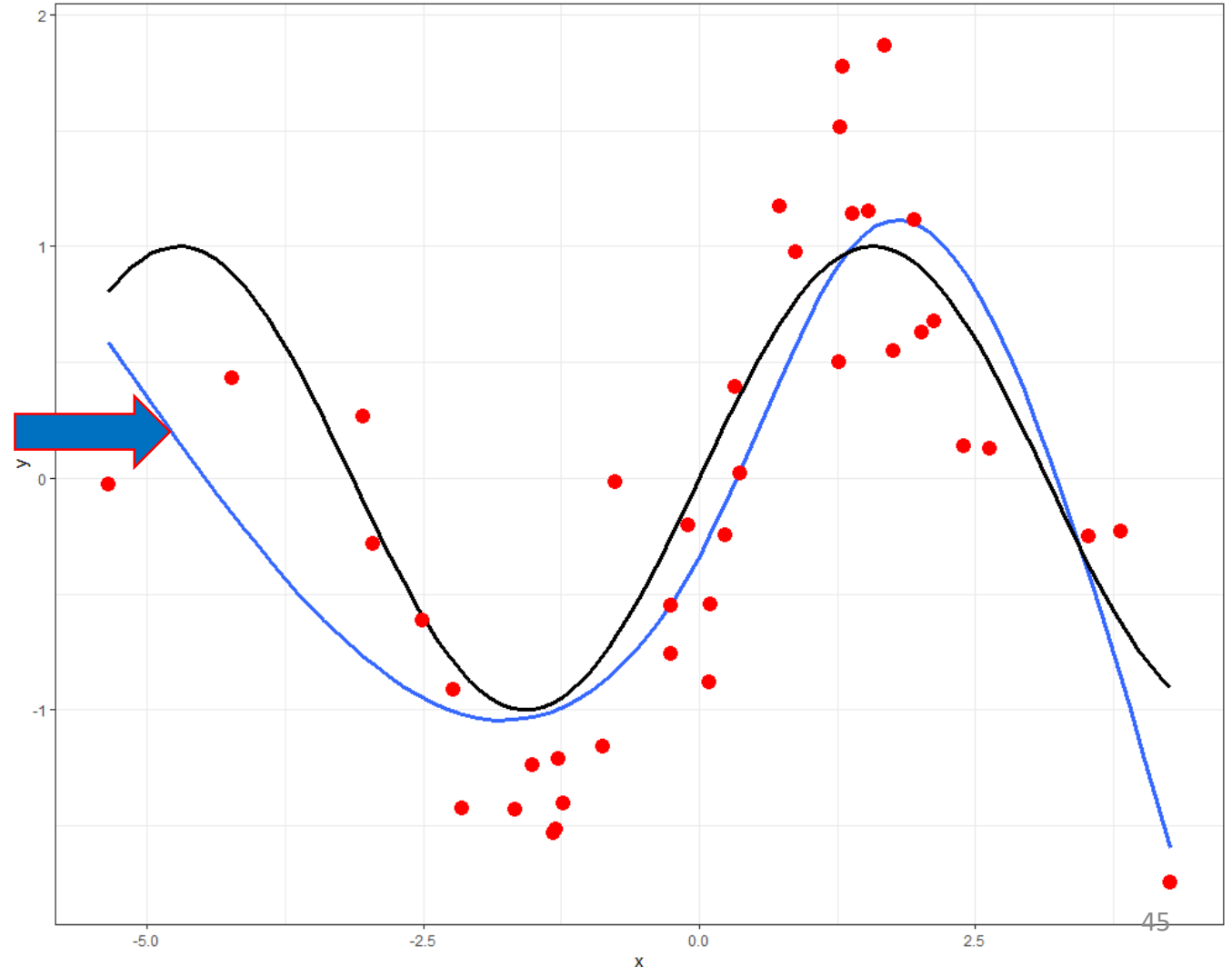
Include a 4 degree-of-freedom spline in our visualization via the `geom_smooth()` geometric object

```
demo_sine %>%  
  ggplot(mapping = aes(x = x, y = y)) +  
    geom_smooth(formula = y ~ splines::ns(x, 4),  
               method = "lm",  
               se = FALSE,  
               size = 1.35) +  
    stat_function(fun = sin,  
                 color = "black", size = 1.35) +  
    geom_point(size = 4, color = "red") +  
    theme_bw()
```



Include a 4 degree-of-freedom spline in our visualization via the `geom_smooth()` geometric object

```
demo_sine %>%  
  ggplot(mapping = aes(x = x, y = y)) +  
  geom_smooth(formula = y ~ splines::ns(x, 4),  
             method = "lm",  
             se = FALSE,  
             size = 1.35) +  
  stat_function(fun = sin,  
              color = "black", size = 1.35) +  
  geom_point(size = 4, color = "red") +  
  theme_bw()
```



How does a spline SMOOTH the data?

- How many coefficients are estimated to fit a 4 degree-of-freedom natural spline?
- Let's check by calling `lm()`:

```
> mod_ns04_mle <- lm( y ~ splines::ns(x, 4), data = demo_sine )
>
> mod_ns04_mle %>% broom::tidy()
# A tibble: 5 × 5
```

	term <chr>	estimate <dbl>	std.error <dbl>	statistic <dbl>	p.value <dbl>
1	(Intercept)	0.584	0.349	1.67	0.103
2	splines::ns(x, 4)1	-1.05	0.395	-2.66	0.0117
3	splines::ns(x, 4)2	2.36	0.356	6.62	0.000000103
4	splines::ns(x, 4)3	-2.57	0.848	-3.03	0.00446
5	splines::ns(x, 4)4	-1.04	0.418	-2.48	0.0180

Coefficients (slopes) MULTIPLY features...but what features does the SPLINE use?

- Remember **DESIGN MATRIX** organizes the features of the model!
- I recommend creating the design matrix **BEFORE** fitting models so you can check the **NUMBER of features** associated with the model!!

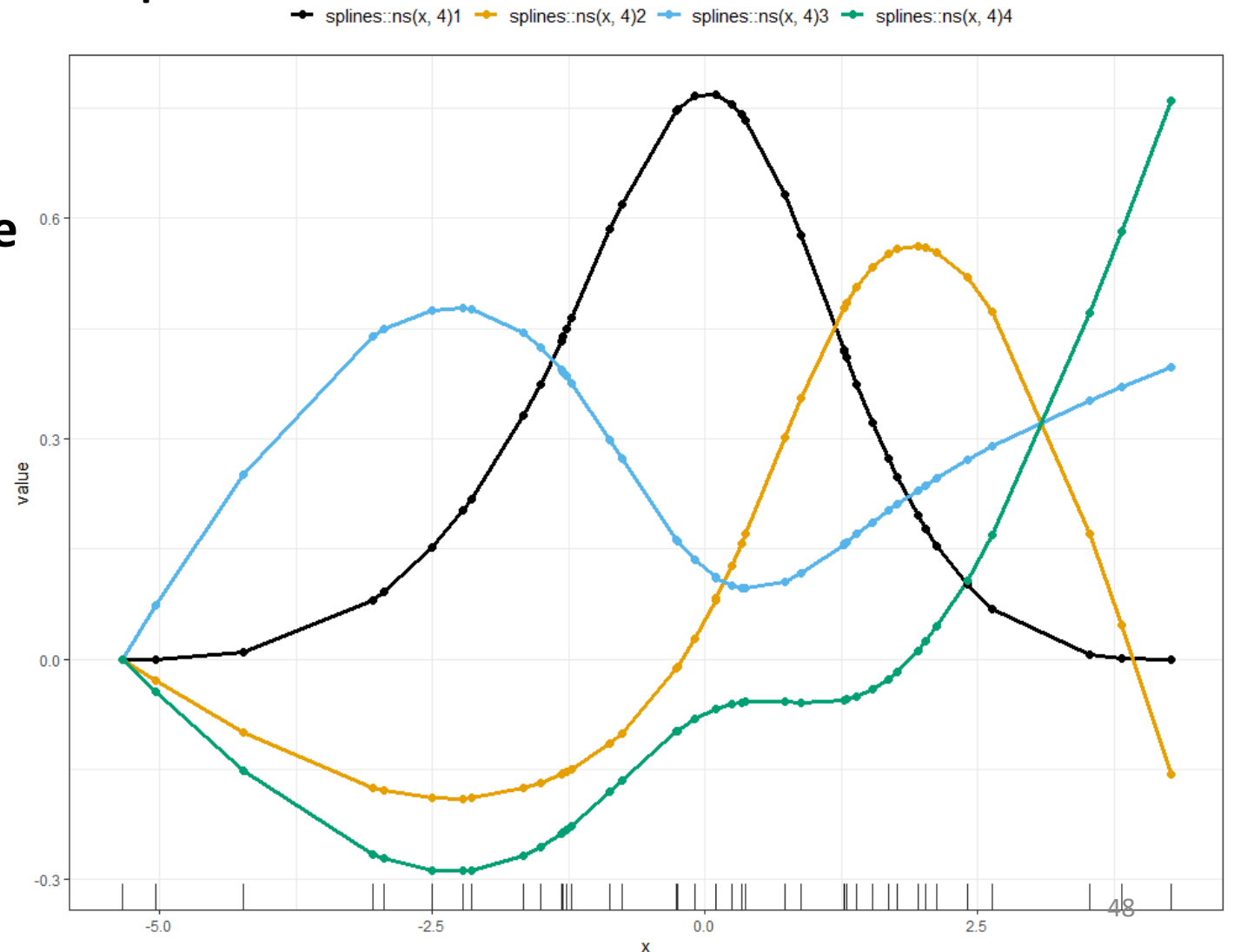
```
> Xmat4 <- model.matrix(y ~ splines::ns(x, 4), data = demo_sine)
>
> head(Xmat4)
```

	(Intercept)	splines::ns(x, 4)1	splines::ns(x, 4)2	splines::ns(x, 4)3	splines::ns(x, 4)4
1	1	0.5847695	-0.11432760	0.2985161	-0.18015653
2	1	0.7692863	0.07995454	0.1118044	-0.06747475
3	1	0.2172354	-0.18885693	0.4763184	-0.28746151
4	1	0.4101723	0.48468074	0.1595724	-0.05488827
5	1	0.4640323	-0.14910265	0.3761530	-0.22701095
6	1	0.1021646	0.51929650	0.2708740	0.10766493

What do the columns or FEATURES of the basis design matrix represent?

Each spline “feature” represents a polynomial relationship over the range of the input x .

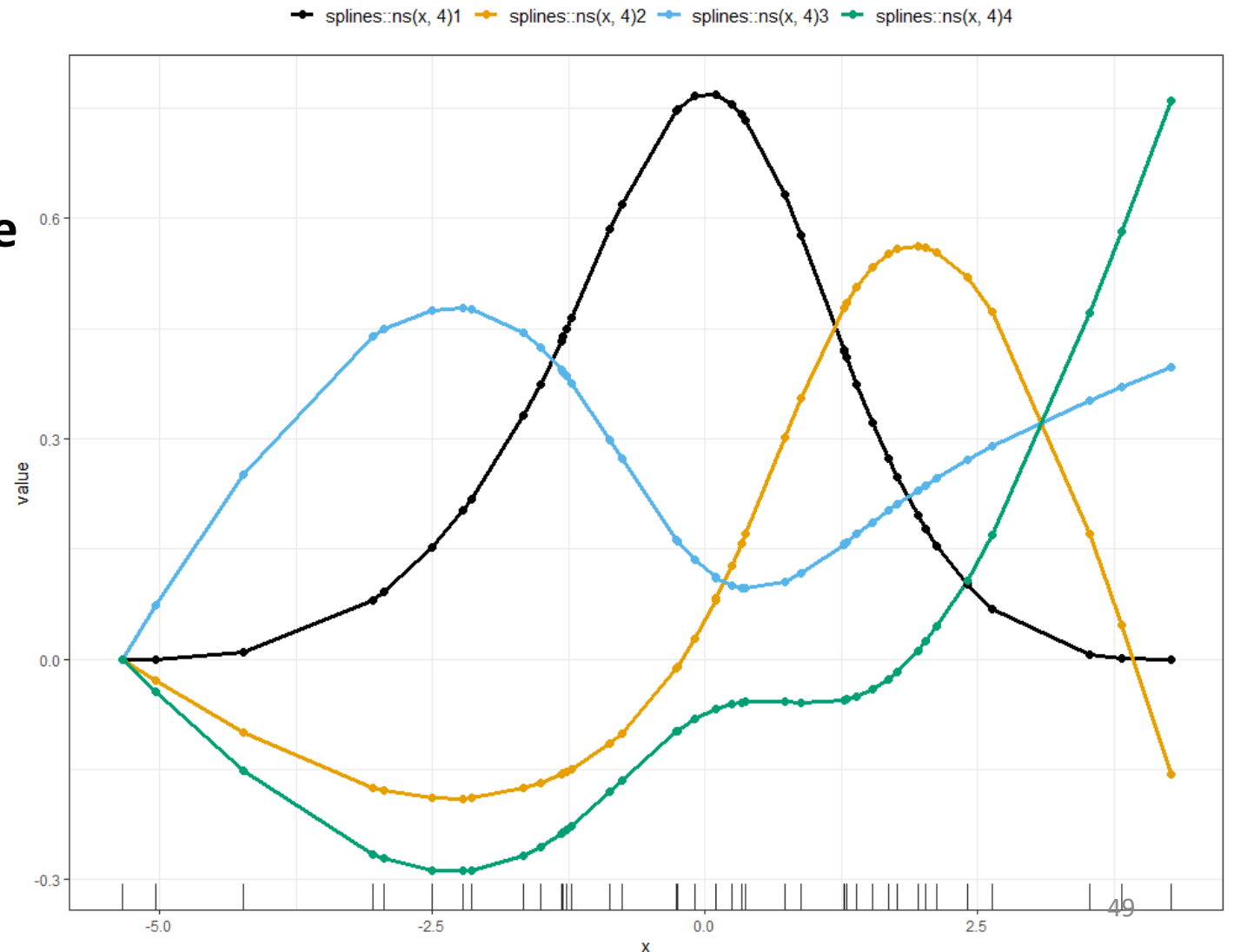
With 4 degrees-of-freedom, we have 4 spline “features”!



How do we use these FEATURES derived from the input?

Each spline “feature” represents a polynomial relationship over the range of the input x .

With 4 degrees-of-freedom, we have 4 spline “features”!



We do NOT SUM the features together!

$$\mu_n \neq s_1(x_n) + s_2(x_n) + s_3(x_n) + s_4(x_n)$$

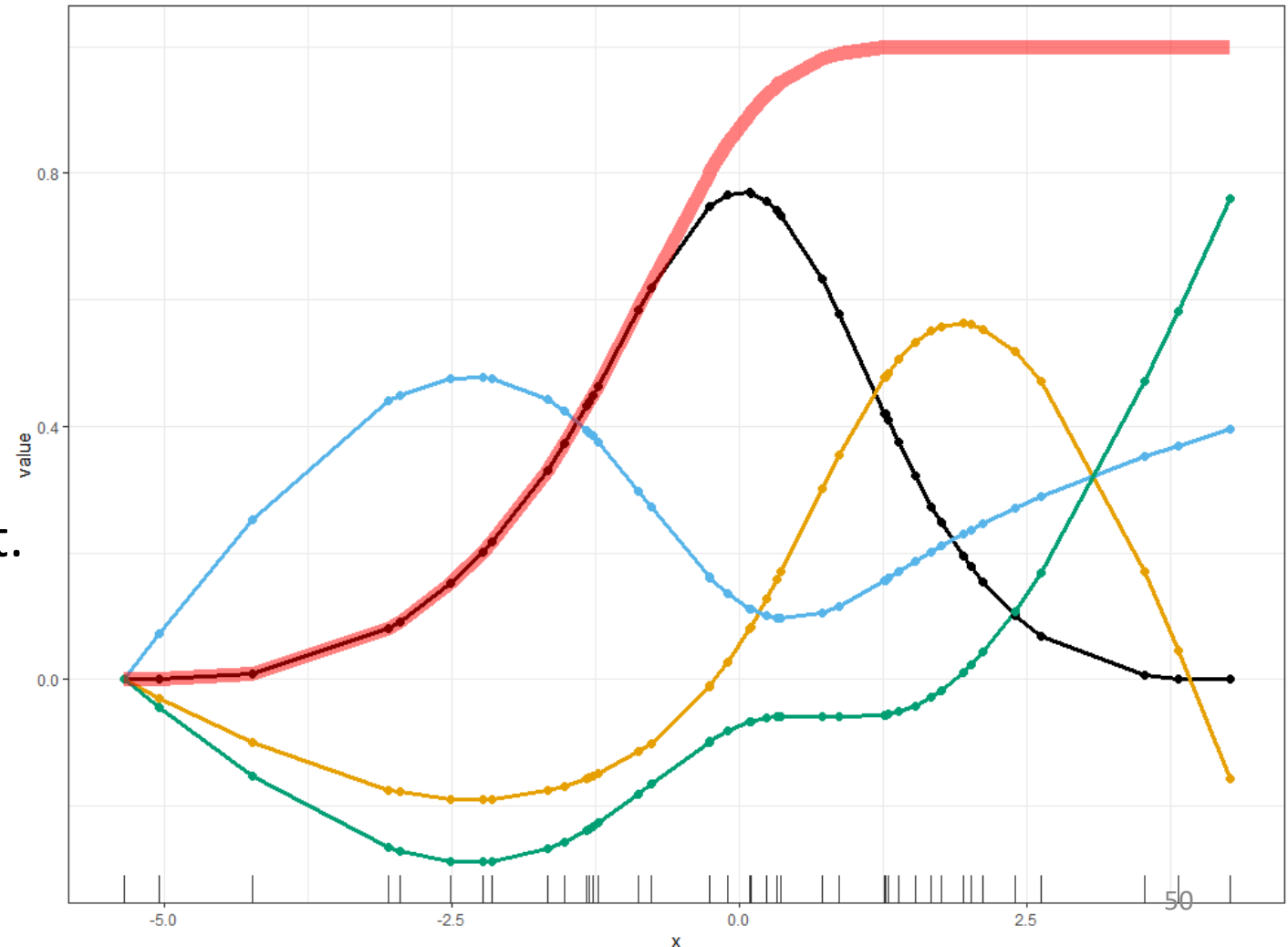
Where: $s_j = \text{splines} :: \text{ns}(x, 4)j$

— splines::ns(x, 4)1 — splines::ns(x, 4)2 — splines::ns(x, 4)3 — splines::ns(x, 4)4

If we summed the 4 FEATURES
(polynomials) at each input value...

The resulting trend is NOT flexible...

The resulting trend corresponds to the
red S-curve like curve shown to the right.



Instead, we use a WEIGHTED SUM of the features!

$$\mu_n = \beta_0 + \beta_1 s_1(x_n) + \beta_2 s_2(x_n) + \beta_3 s_3(x_n) + \beta_4 s_4(x_n)$$

Where: $s_j = \text{splines} :: \text{ns}(x, 4)j$

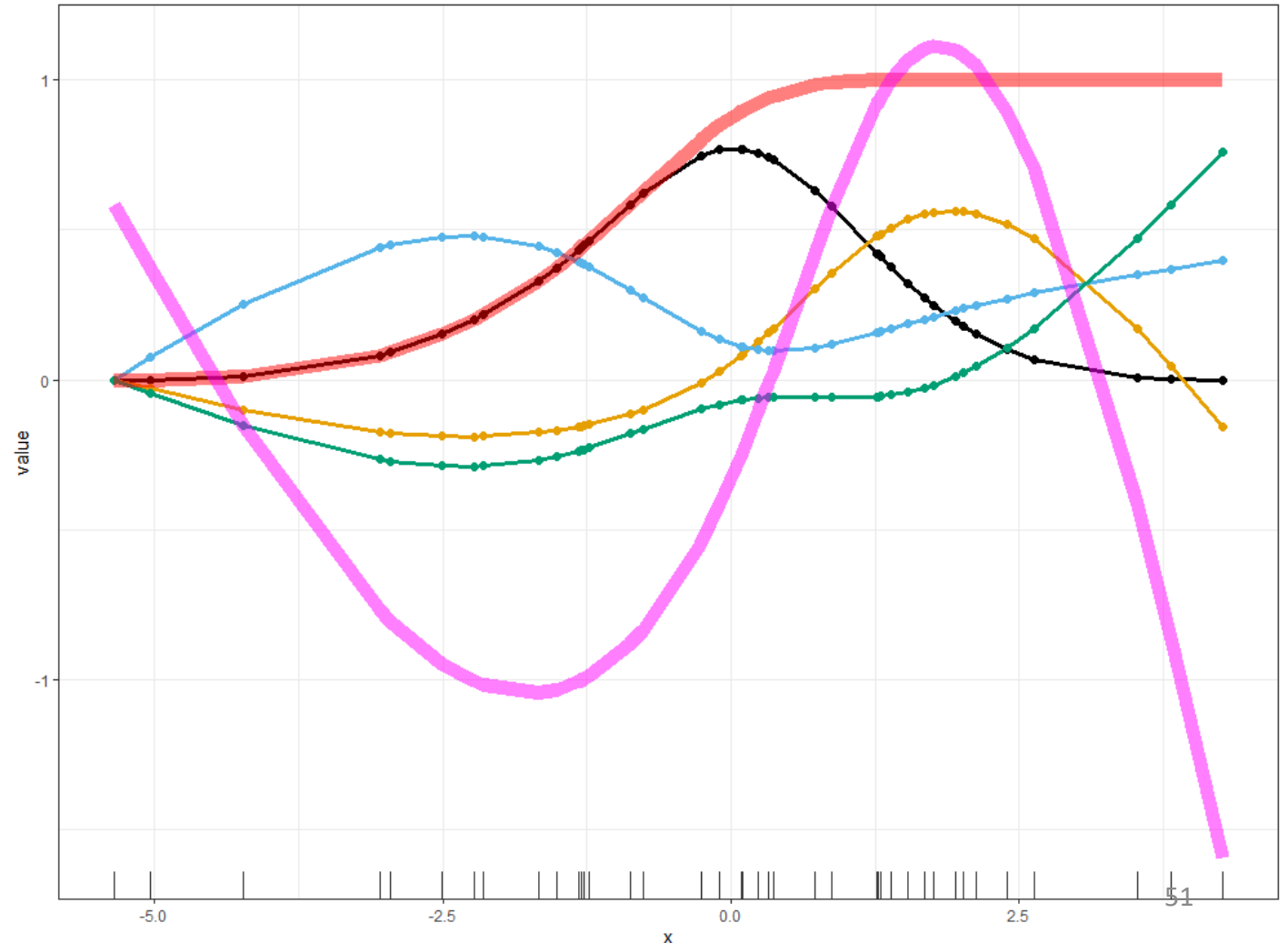
— splines::ns(x, 4)1 — splines::ns(x, 4)2 — splines::ns(x, 4)3 — splines::ns(x, 4)4

Each FEATURE is a polynomial over the range of the input x .

The β -parameters we must learn act as WEIGHTS applied to those polynomials.

The greater the absolute value of the weight, the greater the impact the polynomial has on the mean trend.

The WEIGHTS or REGRESSION COEFFICIENTS allow the SPLINE FEATURES to capture the NON-LINEAR output-to-input RELATIONSHIP!



What happens if we use MORE degrees-of-freedom?

For example, let's consider a 9 degree-of-freedom natural spline.

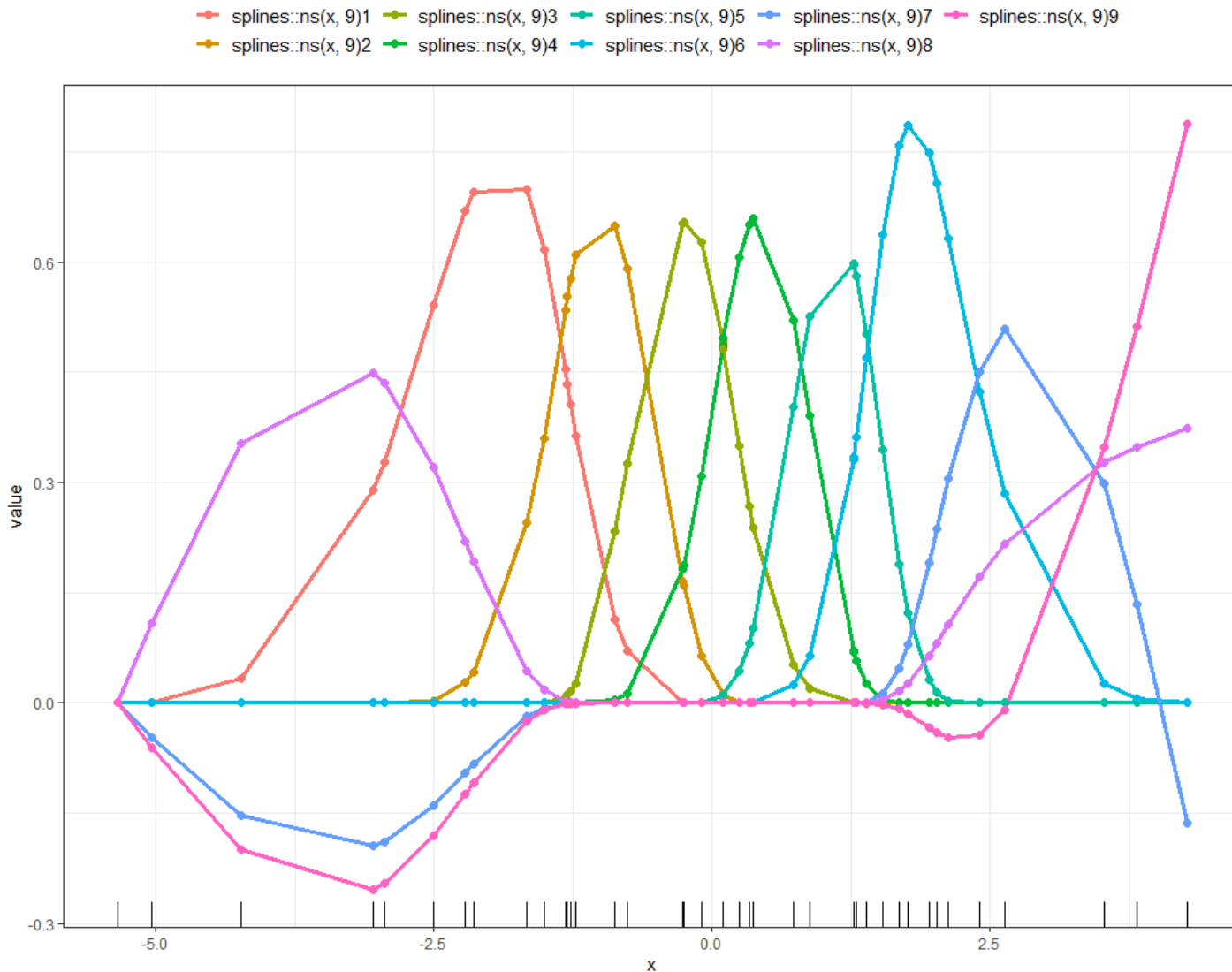
Before fitting the model, let's create the basis DESIGN MATRIX:

```
> Xmat9 <- model.matrix(y ~ splines::ns(x, 9), data = demo_sine)
> dim(Xmat9)
[1] 41 10
> colnames(Xmat9)
[1] "(Intercept)"      "splines::ns(x, 9)1" "splines::ns(x, 9)2" "splines::ns(x, 9)3" "splines::ns(x, 9)4" "splines::ns(x, 9)5" "splines::ns(x, 9)6"
[8] "splines::ns(x, 9)7" "splines::ns(x, 9)8" "splines::ns(x, 9)9"
```

There are 10 columns because there are 9 spline FEATURES and the INTERCEPT column of 1s!!

The model therefore has 10 regression coefficients to estimate!!

The features are still polynomials...
We just have more of them!!!

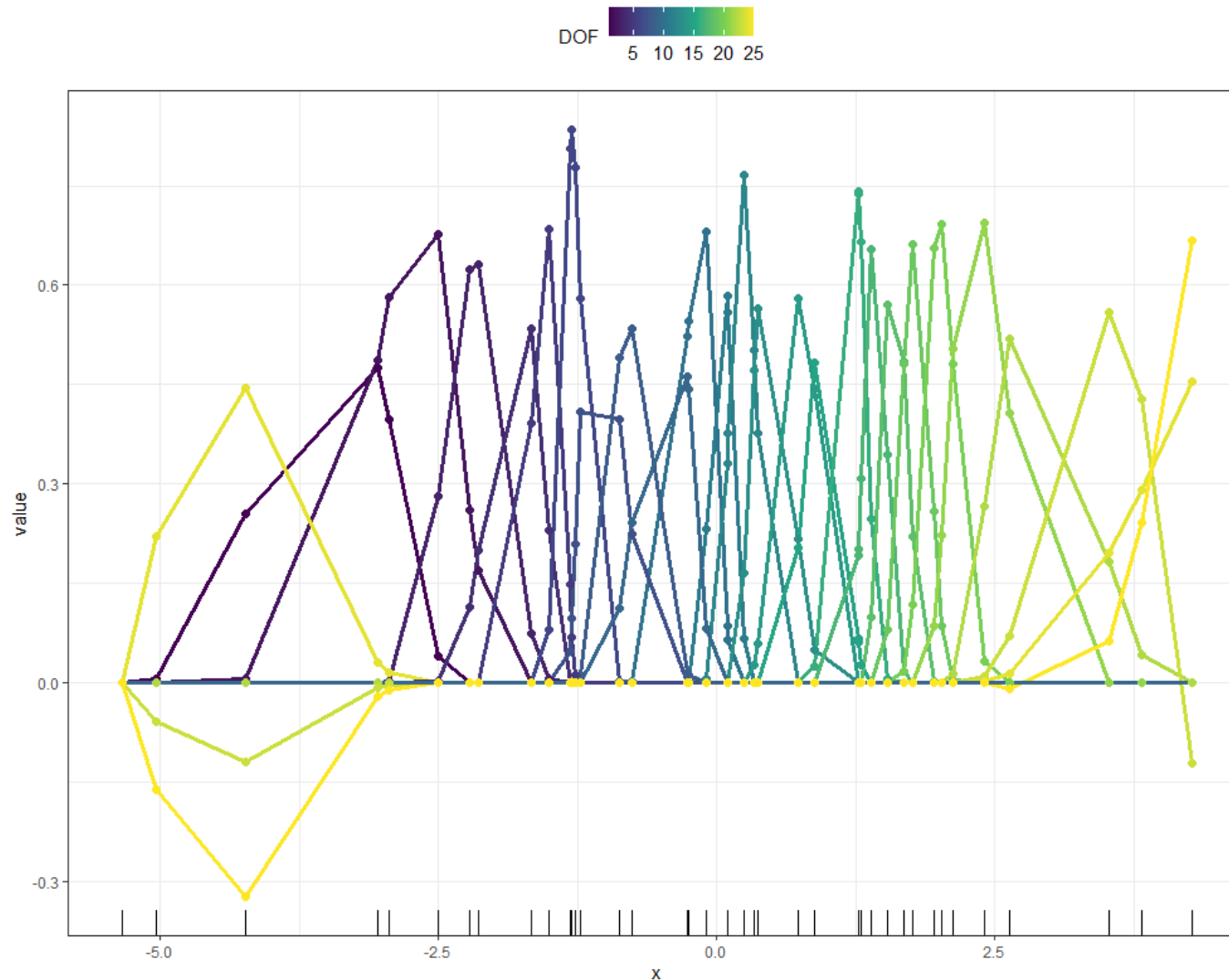


We can try even higher degrees-of-freedom...
Why not try 25 degrees-of-freedom?

```
> Xmat25 <- model.matrix(y ~ splines::ns(x, 25), data = demo_sine)
> dim(Xmat25)
[1] 41 26
> colnames(Xmat25)
[1] "(Intercept)"      "splines::ns(x, 25)1" "splines::ns(x, 25)2"
[4] "splines::ns(x, 25)3" "splines::ns(x, 25)4" "splines::ns(x, 25)5"
[7] "splines::ns(x, 25)6" "splines::ns(x, 25)7" "splines::ns(x, 25)8"
[10] "splines::ns(x, 25)9" "splines::ns(x, 25)10" "splines::ns(x, 25)11"
[13] "splines::ns(x, 25)12" "splines::ns(x, 25)13" "splines::ns(x, 25)14"
[16] "splines::ns(x, 25)15" "splines::ns(x, 25)16" "splines::ns(x, 25)17"
[19] "splines::ns(x, 25)18" "splines::ns(x, 25)19" "splines::ns(x, 25)20"
[22] "splines::ns(x, 25)21" "splines::ns(x, 25)22" "splines::ns(x, 25)23"
[25] "splines::ns(x, 25)24" "splines::ns(x, 25)25"
```

There are now 26 regression coefficients to estimate!!!

The basis FEATURES associated with the 25 degree-of-freedom model look crazy...



We have examined the features associated with 4, 9, and 25 degree-of-freedom natural splines

- Which degree-of-freedom is the best?

